3D Space Partitioning Viewer

Computer Graphics (E016712A)

Antonio Romero Martín Siebe Ternest

May 1, 2026

Contents

1 Introduction **3**

2 Reading the Input Data **3**
2.1 Datasets 4

3 Space Partitioning Algorithms **5**
3.1 Octree 5
3.2 K-d Tree 6
3.3 BSP Tree 9

4 Viewer **11**
4.1 Rendering Architecture 11
4.2 Coloring 12
4.3 BSP Visualization 12
4.4 UI and Dataset Loading 13

5 Comparative Analysis **14**
5.1 Main Differences between the Three Trees 14
5.2 Benefits of an Octree over a BSP-Tree 15
5.3 Ray Intersection Queries 15
5.4 Dynamic Scenes 15
5.5 Urban Scenes 16

6 Conclusion **16**

1 Introduction

Efficiently organizing millions of 3D points is a fundamental challenge in computer graphics and spatial computing. Space partitioning data structures address this by hierarchically subdividing 3D space, enabling fast queries such as ray intersection, frustum culling, and nearest-neighbour search. As LiDAR scanners become ubiquitous in autonomous driving, urban mapping, and robotics, the choice of partitioning structure has direct consequences for rendering performance, memory usage, and query latency at scales ranging from thousands to tens of millions of points.

This report documents the design decisions, implementation challenges, and comparative analysis of a 3D space partitioning viewer built for LiDAR point clouds. Three hierarchical data structures were implemented and compared: an **Octree**, a **k-d Tree**, and a **Binary Space Partitioning (BSP) Tree**. All three were evaluated on three real-world LiDAR datasets – a small indoor corridor scan, a mid-size UFO-shaped structure, and the city of Hasselt (∼19.6 million points) – covering a wide range of point densities and spatial configurations.

The viewer is built with **Three.js** (WebGL), which provides scene-graph management, instanced geometry, and camera controls out of the box, allowing the implementation effort to focus on algorithm design and rendering architecture rather than low-level shader infrastructure. The project is structured as a single-page web application served by Vite, with all three algorithms in a shared `src/algorithms/` module and all visualization logic in `src/viewer/`.

Section 2 describes the input data pipeline and PCD file parsing. Section 3 presents each algorithm's design decisions, complexity, and memory characteristics. Section 4 documents the viewer and rendering architecture. Section 5 provides a comparative analysis across build cost, query performance, and suitability for different scene types.

2 Reading the Input Data

(Both authors)

Input datasets are PCD (Point Cloud Data) files from LiDAR scanners. The module `src/loaders/pcdLoader.js` implements a **fast-path parser** for the layout shared by all three bundled datasets: `FIELDS x y z`, `SIZE 4 4 4`, `TYPE F F F`, `DATA binary`. The parser reads the ASCII header up to the `DATA binary\n` boundary marker and then wraps the remaining raw bytes directly in a `Float32Array` view, bypassing the field-by-field `DataView.getFloat32()` reads that Three.js `PCDLoader` would otherwise perform. For Hasselt (∼19.6 million points, 235 MB) this eliminates ∼58.8 million individual `DataView` calls, reducing the post-download parsing phase from several seconds to milliseconds. ASCII files and binary PCD with extra fields or non-float types fall back to Three.js `PCDLoader` automatically.

The module unifies three loading paths behind a single interface:

- **Bundled datasets** (UFO, Corridor, Hasselt) – served as static assets by the development server; the URL is fetched via XHR (native progress events) and the resulting `ArrayBuffer` is handed to the fast-path parser.

- **Arbitrary URL** – the control panel exposes a URL input field; any `.pcd` URL entered by the user follows the same XHR path and is cached by URL string after the first load.
- **Local files** – `.pcd` files dropped onto the viewer or selected via the file picker arrive as browser `File` objects; `File.arrayBuffer()` is used to obtain the raw bytes directly, which are then passed to the same parser. Local files are cached under the key `__local__:<filename>`, so the user can switch to a different dataset and return to the local file instantly without re-reading it.

Once a point cloud is loaded, `datasetService` performs two passes over the position buffer. First, a **Z-range scan** finds the global minimum and maximum Z values using a direct typed-array loop rather than `Box3.setFromObject`, avoiding Three.js method-call overhead for each of the 19.6 million points. Second, a **colorization pass** maps each point's Z coordinate linearly to an HSL hue in the range [0.7 (blue), 0.0 (red)] and writes it into a `Float32Array` color attribute. This per-point color is used as the baseline appearance for all three algorithms. All loaded datasets are **cached after the first load**: the colorized `Points` object, the bounding-box extents, and the camera target are stored – URL-based datasets under their URL string, local files under `__local__:<filename>` – so switching between algorithms, adjusting depth, or returning to a previously loaded dataset never triggers a reload or a second colorization pass over millions of points.

2.1 Datasets

Three binary PCD files were used throughout this project as the primary evaluation datasets; the viewer also accepts any user-supplied `.pcd` file via drag-and-drop or URL input. Table 1 summarises the key properties of the three bundled datasets, derived directly from the file headers and bounding-box statistics.

Dataset	Points	Size	X span (m)	Y span (m)	Z span (m)
Corridor	1,622,607	19.5 MB	86	70	12
UFO	10,750,846	129 MB	193	91	37
Hasselt	19,615,568	235 MB	385	368	24

Table 1: Dataset properties extracted from PCD headers and bounding-box sampling.

Corridor is the smallest dataset. Its narrow Z extent (12 m) relative to its horizontal footprint is consistent with a ground-level indoor or semi-outdoor corridor scan, where the sensor sweeps a bounded vertical range.

UFO is a mid-size outdoor scan (∼10.8 million points). The horizontal extents are moderate (∼200 m) and the Z span (37 m) is the largest of the three, reflecting a more varied vertical structure than the other two datasets.

Hasselt is the largest dataset (∼19.6 million points, 235 MB). Its footprint is roughly 385×368 m but the Z extent is only 24 m, which is characteristic of a flat urban outdoor scan – dense near facades and the ground plane, sparse in open airspace. This dataset was used for all stress-testing of memory usage and build time.

3 Space Partitioning Algorithms

All three implementations share a common base class (`BaseTree`) that establishes a key memory design: throughout construction every algorithm works exclusively with **integer indices** into the original `Float32Array` rather than allocating one `Vector3` object per point. Each index is a four-byte integer; coordinates are read inline as `positions[idx*3]`, `positions[idx*3+1]`, `positions[idx*3+2]`. For Hasselt (∼19.6 million points) the construction index array consumes ∼78 MB, compared to the ∼784 MB that a `Vector3[]` approach would require (roughly 40 bytes per JavaScript object versus 4 bytes per index). The global bounding box is computed by the shared helper `_computeBoundsFromPositions`, which scans the `Float32Array` in a single pass and creates only two `Vector3` objects (the AABB corners) regardless of point count. After each tree is fully built, the algorithm-specific index structures and the reference to the positions buffer are nulled, retaining only the node structure in memory.

3.1 Octree

(Both authors)

Design decisions

The tree is built top-down: starting from a global bounding box, it is recursively subdivided into 8 equal octants. A node stops splitting when either `maxDepth` is reached *or* its point count falls at or below `maxPointsPerNode` – whichever comes first.

The first design decision was to **force the root bounding box into a perfect cube** by recentering it on the bounding-box centroid and expanding all three half-extents to half of the longest side. This guarantees that *every voxel at a given depth has the same side length*, regardless of the dataset's spatial extent. The consequences are threefold: the leaf-size readout in the UI is meaningful because all leaves at the same depth are equally sized; ray-AABB intersection tests benefit from uniform cell dimensions that allow precomputed slab distances; and level-of-detail rendering is straightforward – querying a shallower depth uniformly coarsens the whole scene at the same rate in every direction.

A second decision was **pre-allocating eight point-bucket arrays per depth level** instead of creating them inside each recursive call. The naïve approach would allocate eight new arrays per internal node; across thousands of recursive calls on large datasets this generates significant garbage-collection pressure. With the pre-allocated table each bucket is reset to length zero via `.length = 0` and reused, keeping allocation overhead constant regardless of tree size.

Each point is assigned to its octant using a **3-bit index**: bit 0 is set if the X coordinate $\geq$ `centerX`, bit 1 for Y, bit 2 for Z. The same index drives the child bounding-box computation in $O(1)$ via bitwise conditions, keeping the assignment and the geometry fully consistent without any per-axis branching.

Only non-empty octants produce child nodes; technically, this makes the implementation a **sparse octree** (up to 8 children per node, not always 8). This is crucial for large, non-uniform point clouds such as Hasselt, where most of the airspace above the city produces empty octants – skipping them avoids wasting memory and traversal time, and ensures

only occupied regions are rendered.

Complexity and memory

- **Build:** $O(n \log n)$ – the split plane (geometric midpoint) is computed in $O(1)$; classifying each point into an octant requires three comparisons ($O(1)$ per point). Each of the n points is touched once per depth level, across $O(\log n)$ levels.
- **Single-point update:** $O(\log n)$ – follow the path from root to leaf and add/remove the point from the appropriate voxel; the split planes never change.
- **Memory:** $O(n)$ asymptotically – at most n leaf nodes at maximum depth; internal nodes null their index arrays once split. The dominant allocation during construction is the integer index array ($\sim$78 MB for Hasselt), which is freed once the build completes, along with the positions reference and the pre-allocated bucket table. The retained node structure size scales with the number of occupied octants, which depends on the dataset's spatial distribution and the chosen depth.

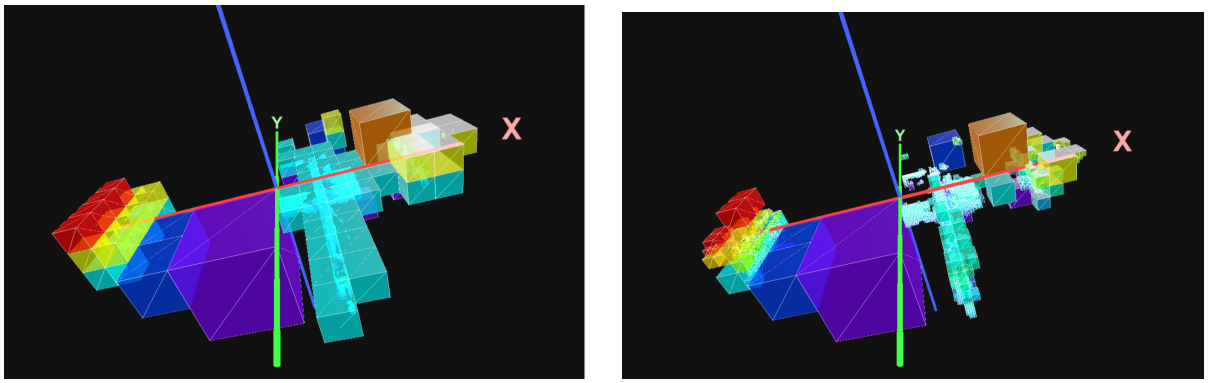

Figure 1: Octree on the Corridor dataset at a low depth (left) and a high depth (right). Color encodes the Z coordinate (blue = low, red = high). The wireframe outlines show voxel boundaries.

3.2 K-d Tree

(Both authors)

Design decisions

The split plane is always placed at the **median** of the active coordinate. Using the median rather than the geometric midpoint is the key choice: it guarantees that each split divides the point set into two roughly equal halves regardless of point density, keeping the tree balanced.

Axis-selection strategies. Three strategies are implemented and selectable at run time via the **Split Mode** control in the viewer:

- **Cycle** (default): the axis rotates X $\rightarrow$ Y $\rightarrow$ Z with depth (`depth % 3`). Simple and data-independent; all nodes at the same depth always split on the same axis.

- **Widest**: picks the axis whose bounding-box extent is largest at the current node, using the tight AABB already computed during construction. $O(1)$ per node; ensures the longest dimension is always halved first, producing more cube-like cells and reducing long thin partitions.
- **Variance**: picks the axis with the highest variance of the point coordinates – the dominant diagonal entry of the covariance matrix, equivalent to what a full PCA would identify as the principal axis of spread when restricted to axis-aligned splits. Variance is estimated via **Welford's online algorithm** on a stride sample of up to 1 024 points, keeping per-node cost $O(1)$ in practice while remaining numerically stable for coordinates of any scale. Unlike *widest*, this strategy responds to the actual distribution of points rather than just the bounding geometry.

A module-level scratch `Vector3` is reused across all *widest* computations to avoid per-call heap allocations.

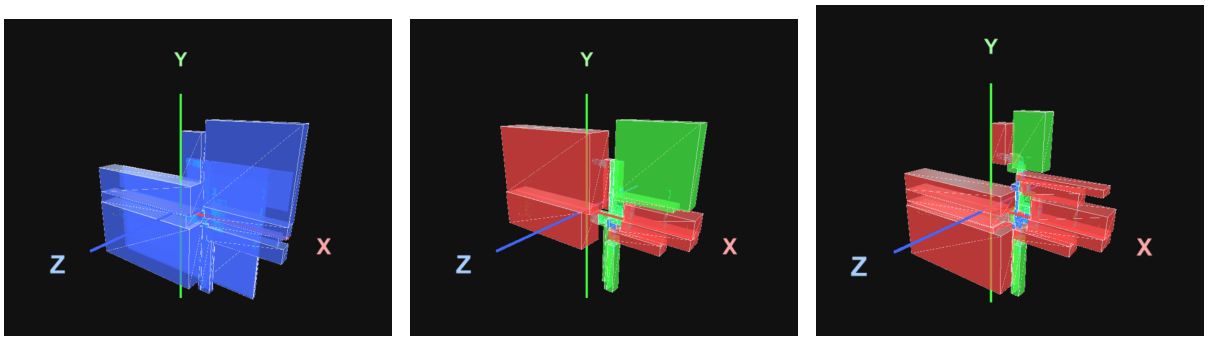

Figure 2: K-d Tree split mode comparison on the Corridor dataset: Cycle (left), Widest (center), Variance (right). In Cycle mode all cells at a given depth share the same color; in Widest and Variance the colors are mixed, reflecting data-driven axis choices at each node.

Median selection. Finding the median without a full sort is done with a **3-way quickselect** (Dutch National Flag partitioning). This is $O(n)$ average per node and handles duplicate coordinates correctly by grouping equal-valued points into a pivot zone in a single pass, so repeated coordinates in the cloud do not degrade performance to $O(n^2)$ as they would with a naïve quicksort.

All points are stored in a **single shared array** that is partitioned in-place during construction. Each node records a `[start, end)` range into this array – no per-node point copies are made. Bounding boxes are computed from the actual point distribution after partitioning, giving tight cuboids instead of the oversized AABB of the raw split range.

Balancing

Because every split is at the median, the k-d tree is **structurally balanced**: each partition divides the point set into two equal-count halves, bounding the tree depth at $\lceil \log_2 n \rceil$. In practice, however, the `maxPointsPerNode` leaf threshold means that a sparse region can become a leaf several levels earlier than a dense region that keeps splitting. The tree therefore remains **well-balanced** – traversal depth stays close to $O(\log n)$ – but the strict

guarantee that all leaves sit at the same depth does not hold when the leaf threshold is active.

A degenerate edge case is caught explicitly: after quickselect, if the left partition would be empty (`medianIndex == start`), the node is finalized as a leaf immediately rather than recursing into a zero-size split. This guards against configurations where the leaf threshold is zero and a single remaining point would otherwise produce an empty left child.

A balanced tree is especially important for ray traversal: in a balanced tree the worst-case traversal depth is $O(\log n)$, whereas an unbalanced tree can degrade to $O(n)$.

Complexity and memory

- **Build:** $O(n \log n)$ – quickselect is $O(n)$ per node; there are $O(\log n)$ levels; total work per level is $O(n)$.
- **Query / ray intersection:** $O(\log n)$ average due to balance.
- **Single-point update:** $O(\log n)$ for search; however, naïve insertion without rebalancing degrades the tree over time.
- **Memory:** $O(n)$ during construction (the shared `Int32Array` of n indices, $\sim$78 MB for Hasselt). After build both the index array and the positions reference are freed. The retained structure has at most $2^{\texttt{maxDepth}+1}-1$ nodes; for the depth-8 configuration used here that is $\leq$511 nodes ($\sim$100 kB), so all three cached variants (Cycle, Widest, Variance) coexist in memory without retaining any copy of the point cloud.

Memory problem encountered. An initial prototype represented each point as a `Vector3` JavaScript object. Each object carries a heap overhead of roughly 40–80 bytes on top of the 12 bytes of coordinate data; the construction array for Hasselt therefore consumed $\sim$784 MB–1.5 GB, exceeding the browser heap limit and causing an out-of-memory crash even for a single tree build.

The solution was to replace the `Vector3[]` array with a single `Int32Array` of n integer indices into the original `Float32Array`. At 4 bytes per index the construction array for Hasselt drops to $\sim$78 MB. After the tree is fully built both the index array and the positions reference are nulled, leaving each cached variant at $\sim$100 kB. Additionally, the Variance axis-selection computation was capped at a stride sample of 1 024 points per node: without the stride, each node at the top of the tree would scan all of its $O(n)$ points, making the per-node *cost* $O(n)$ rather than $O(1)$. The stride bounds it to at most 1 024 iterations per node regardless of subtree size. After these changes all three Hasselt variants coexist in memory without issue.

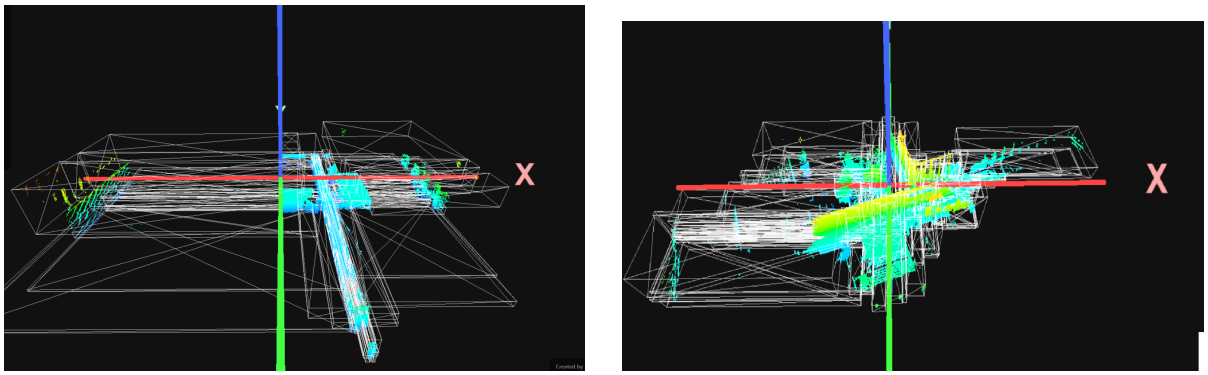

Figure 3: K-d Tree on the Corridor dataset (left) and the UFO dataset (right). The cuboids are non-uniform because the median split follows the actual point distribution, not a fixed midpoint.

3.3 BSP Tree

(Antonio Romero Martín)

Design decisions

A BSP tree splits space with **arbitrarily oriented planes**. The central design challenge is choosing a good plane at each node. We evaluated three strategies:

1. **Random plane** – fast but produces uneven splits.
2. **Axis-aligned median** – equivalent to k-d tree; does not exploit non-axis-aligned structure.
3. **PCA-based plane** (Principal Component Analysis) – the plane normal is the dominant eigenvector of the covariance matrix of the node's points; the plane is positioned at the **median projection** of the node's points onto that axis.

We chose PCA because it adapts to the local geometry: in a corridor the principal axis aligns along the corridor, producing long thin cells; in an open area the plane follows the dominant spread of points. This reduces the likelihood of degenerate splits compared to random or axis-aligned strategies.

The eigenvector is found by **power iteration** (up to 20 iterations on the 3×3 covariance matrix, with early termination when the vector magnitude falls below 10^{-10}), which converges reliably for real-world point clouds. The covariance matrix is computed in a **single pass** using **Welford's online algorithm**: for each point the running mean is updated first, then all six covariance entries are accumulated as $\Delta_{\text{before}} \times (\text{value} - \mu_{\text{new}})$. No scratch buffers are required; only six scalar accumulators are maintained alongside the three mean values.

Once the principal axis $\mathbf{n} = (n_x, n_y, n_z)$ and centroid $\mathbf{o}$ are known, every point is projected onto the axis: $d_i = (\mathbf{p}_i - \mathbf{o}) \cdot \mathbf{n}$. The **median projection** d_{med} is found via a **3-way quickselect** (Dutch National Flag) in $O(n)$ average, and the split plane is placed at $\mathbf{o} + d_{\text{med}}\,\mathbf{n}$. Points with $d_i \geq d_{\text{med}}$ go to the front child; the rest go to the back child. This guarantees that each split divides the point set into two equal-count halves, bounding the

tree depth at $\lceil \log_2 n \rceil$ and eliminating the $O(n^2)$ worst case of the centroid-based split.

A node stops splitting when either `maxDepth` is reached *or* its point count falls at or below `maxPointsPerNode`. If the PCA plane produces a degenerate partition – all points falling on the same side – the node is finalized as a leaf immediately to avoid infinite recursion on numerically degenerate inputs.

Why BSP takes longer to build than an Octree

An octree's split plane is always the three axis-aligned midplanes of the current node – computed in $O(1)$ regardless of point count. A BSP node must **examine all its points** to compute the covariance matrix (Welford pass), find the median projection (quickselect pass), and classify points (one more pass): that is $O(n_{\text{node}})$ per node with a larger constant than the Octree. With the median-projection split the tree is balanced and the total work sums to $O(n \log n)$ – the same asymptotic class as the Octree – but the per-node constant is roughly 3× higher. On the Hasselt dataset (∼19.6 million points, 235 MB) we measured BSP taking roughly 3× longer than the Octree.

Complexity and memory

- **Build:** $O(n \log n)$ *guaranteed* – the median projection ensures each split divides the point set in half, bounding the tree depth at $\lceil \log_2 n \rceil$; each level processes $O(n)$ points in total across three $O(n_{\text{node}})$ passes (Welford, quickselect, classify). The $O(n^2)$ worst case of the centroid-based split is eliminated.
- **Single-point update:** $O(\log n)$ to locate the correct leaf (balance is now guaranteed). Insertion is additionally expensive: adding a point can shift the local PCA plane, potentially invalidating the split for that node and requiring a rebuild of the affected subtree.
- **Memory:** $O(n)$ at all times – during construction each recursive call allocates a local `Float32Array` of dot products for its node, freed as the call unwinds. Across the call stack at most $\sim 2n$ elements exist simultaneously (∼156 MB for Hasselt), less than the $3n$ (∼234 MB) of the previous scratch-buffer approach. Unlike the k-d tree, the BSP tree *retains* $O(n)$ memory after build: each leaf stores a `Int32Array` of its point indices, needed by the visualizer for per-cell point recolouring. For Hasselt all leaf arrays together hold ∼78 MB permanently, versus ∼100 kB for the cached k-d tree.

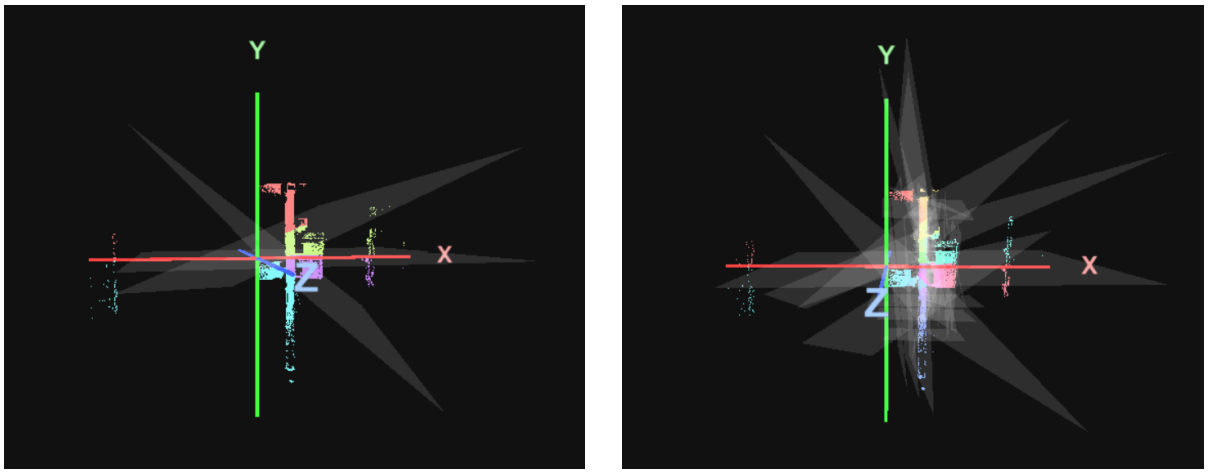

Figure 4: BSP Tree on the Corridor dataset at depth 2 (left) and depth 6 (right). Each cell is assigned a distinct HSL hue; at low depth a few large regions are visible, while at high depth the partitioning follows the fine structure of the point cloud.

4 Viewer

4.1 Rendering Architecture

(Both authors)

The viewer uses a `PerspectiveCamera` (75° FOV) with `OrbitControls` for mouse-driven orbit. Pan and right-click are disabled to keep navigation simple; rotation damping (factor 0.08) smooths the motion without overshoot. On each dataset load the camera is automatically repositioned via a bounding-box fit: a tight axis-aligned bounding box is computed, the longest of its three dimensions is taken, and the camera distance is set to 1.6× that value so the entire point cloud fills the viewport.

Two services decouple concerns cleanly. `datasetService` owns point cloud loading, height-ramp colorization, and camera fitting. `partitionService` owns tree construction, per-variant caching, and depth queries. Trees are built **on first use** and stored under a composite key `algorithm::dataset::variant`. Switching algorithms, changing depth, or toggling between the k-d tree's three split-mode variants is instant once the corresponding tree is cached – no rebuild is triggered on subsequent interactions.

For Octree and k-d Tree, active nodes at the current depth are rendered as **two `InstancedMesh` objects**: a semi-transparent solid box mesh and a wireframe overlay, both backed by a single shared `BoxGeometry`. Using `InstancedMesh` instead of one `Mesh` per node reduces draw calls from thousands to one, which proved essential: without instancing, rendering depth-8 Octree nodes (∼8000 boxes) dropped the frame rate below 10 fps; with instancing it stayed above 60 fps.

Depth navigation. The `+` and `-` keys change the display depth. The tree is queried for nodes exactly at depth d, and leaves shallower than d are also included so coarser resolutions still show all occupied regions. This traversal is O(active nodes) per key press, while the expensive build runs only once.

4.2 Coloring

(Both authors)

Octree cells are colored by the Z coordinate of their bounding-box center using a **height ramp**: HSL hue is linearly mapped from 0.7 (blue/purple, low Z) to 0.0 (red, high Z). This makes the vertical structure of urban point clouds immediately visible and matches the conventional LiDAR viewer palette.

K-d Tree cells are colored by the split axis that created them – that is, the axis of the parent split that produced the boundary between a node and its sibling. The three colors match the on-screen XYZ axis helper exactly: red for X, green for Y, and **blue** for Z. Because both children of a split receive the same axis tag, sibling cells always share the same hue, making the direction of each partition boundary immediately legible without any additional annotation. With the *Cycle* mode all cells at a given depth have the same color; with *Widest* and *Variance* the colors at a given depth can be mixed, directly reflecting the data-driven axis choices made at each node.

Wireframe. Each active node is rendered with a second `InstancedMesh` using wireframe mode, making voxel boundaries visible without obscuring the interior color. Solid fill and wireframe can be toggled independently via checkboxes in the control panel.

4.3 BSP Visualization

(Antonio Romero Martín)

BSP cells are convex polytopes bounded by arbitrary planes, so axis-aligned boxes are not a valid representation. We chose to **recolor the point cloud in place**: each leaf cell is assigned a distinct hue by cycling evenly across the HSL color wheel. This makes cell boundaries visible directly in the data without needing to compute convex hulls (which would be expensive and visually complex for thousands of cells).

Recoloring operates at the GPU buffer level. During construction each BSP leaf records a compact `Int32Array` of the original geometry indices of its points. When BSP mode is activated, the visualizer saves a full copy of the Three.js `BufferGeometry` color attribute (the height-ramp colors applied at load time) and then writes each leaf's HSL color directly into the buffer at the stored indices. When the user switches away from BSP mode, the saved copy is written back, restoring the baseline appearance without reloading the dataset.

Additionally, the **split planes** are shown as semi-transparent quads oriented to the PCA normal at each internal node up to the current depth (Figure 5). Each quad is positioned at the median-projected origin of the split plane (the point $\mathbf{o} + d_{\text{med}}\,\mathbf{n}$ stored in `splitPlane`), scaled to the node's bounding-box size, and oriented via quaternion rotation from the Z axis (the default normal of `PlaneGeometry`) to the split plane normal. This gives an intuitive view of how space is partitioned level by level.

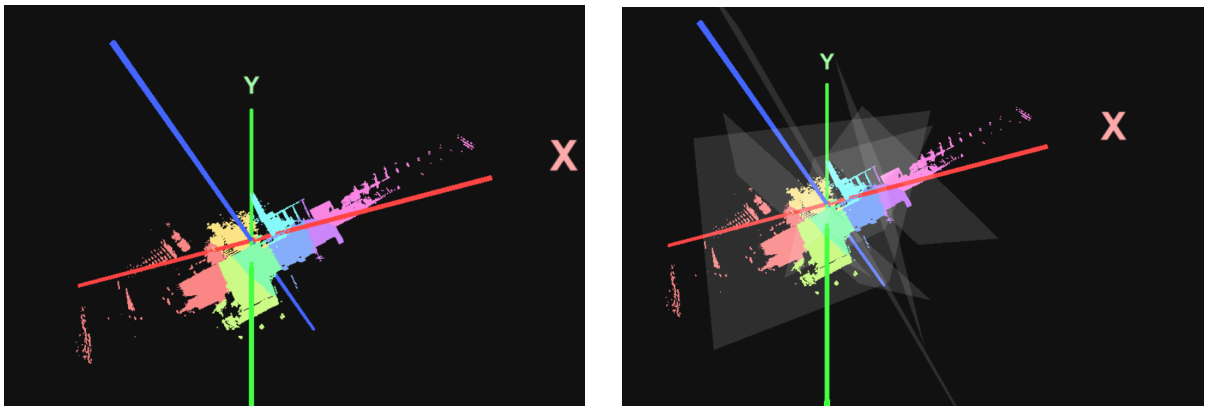

Figure 5: BSP Tree: point cloud colored per leaf cell (left), and split planes shown as semi-transparent quads (right).

4.4 UI and Dataset Loading

(Siebe Ternest)

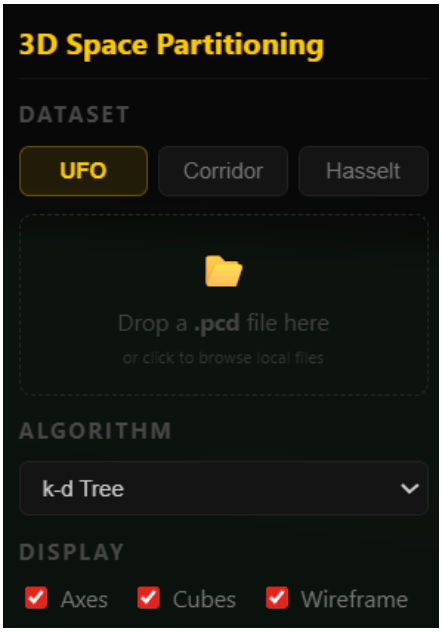

Figure 6: Control panel with algorithm selector, dataset buttons, depth display, leaf-size readout, and visibility toggles.

The control panel (Figure 6) provides algorithm and dataset selection, solid/wireframe visibility toggles, and an XYZ axes toggle. Depth and average leaf size are updated in real time. The **leaf size** readout displays the mean cube root of active-node volumes, $\bar{s} = \frac{1}{N}\sum_{i=1}^{N}\sqrt[3]{V_i}$, giving an intuitive single-number summary of the current partitioning resolution that is comparable across algorithms and datasets.

In addition to the three bundled datasets (UFO, Corridor, Hasselt), users can drag and drop any `.pcd` file onto the viewer. The application runs entirely in the browser; datasets are fetched over HTTP or read from the local file system via the File API.

When the k-d Tree algorithm is selected, a **Split Mode** selector appears in the control panel offering the three axis-selection strategies (Cycle, Widest, Variance). Each variant is cached independently under a composite key `algorithm::dataset::mode`, so switching between modes after the first build is instant. The active mode is highlighted with the same yellow accent used for dataset buttons, making the current strategy immediately visible.

5 Comparative Analysis

5.1 Main Differences between the Three Trees

(Both authors)

	Octree	**K-d Tree**	**BSP Tree**
Split planes per node	3 axis-aligned	1 axis-aligned	1 arbitrary
Children per node	Up to 8	Exactly 2	Exactly 2
Split position	Fixed midpoint	Data median	PCA median projection
Guaranteed balanced	No	Yes (in practice)	Yes
Build cost (constant)	Low	Medium	High
Cell shape	Cube	Cuboid	Convex polytope

The Octree always cuts at the geometric midpoint along all three axes at once, giving $O(1)$ split decisions. The k-d tree cuts one axis at a time at the median, making it self-balancing at the cost of a linear scan. The BSP tree uses an arbitrary plane adapted to the local point geometry, which produces the most informative partitions but incurs the highest build cost.

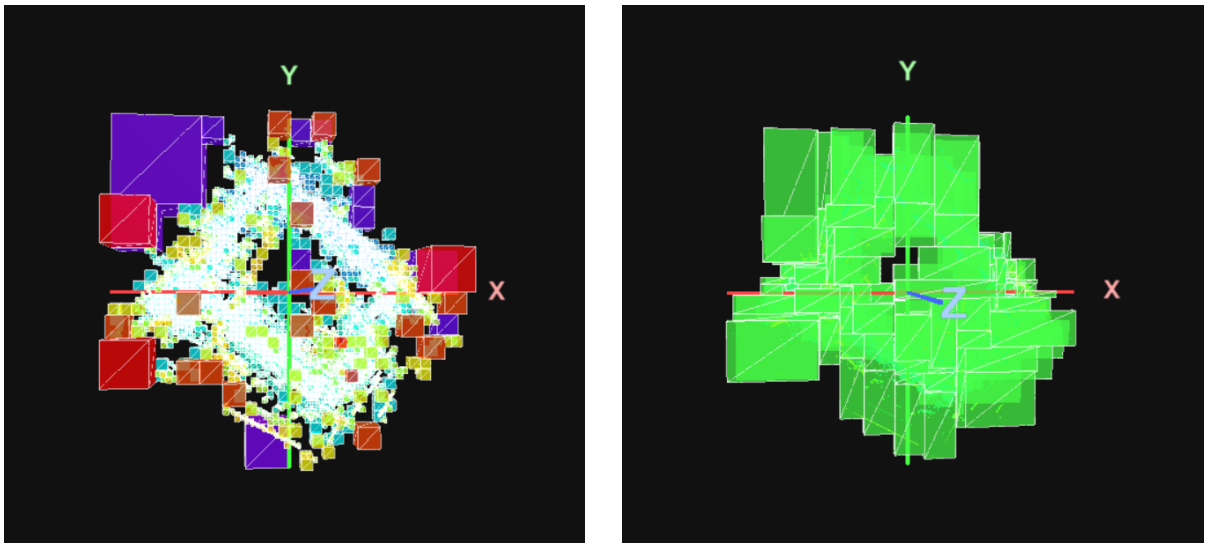

Figure 7: Hasselt dataset (∼19.6 million points): Octree (left) and k-d Tree (right) at comparable depths. The height ramp and axis-color schemes make the structural differences visible at large scale.

5.2 Benefits of an Octree over a BSP-Tree

(Both authors)

- **Simpler, faster construction:** No covariance computation; the split planes are determined in $O(1)$.
- **Easy updates:** Inserting or removing a point only affects the nodes on its root-to-leaf path; split planes never change, so no structural rebuild is needed.
- **Predictable cell shapes:** All cells at a given depth are cubes of identical size, simplifying ray-AABB tests and frustum culling.
- **Implicit empty-space compression:** Only occupied octants create child nodes, so the sparse octree uses no memory for empty regions – a significant advantage for large, non-uniform outdoor point clouds where most of the volume is empty air.

5.3 Ray Intersection Queries

(Both authors)

All three trees support $O(\log n)$ average ray traversal by pruning subtrees whose bounding volumes do not intersect the ray. In practice:

- **Octree:** Ray-AABB intersection (slab method) is very fast with six comparisons per node; the regular grid at each level enables front-to-back ordering, allowing early termination once the first hit is found.
- **K-d Tree:** Each internal node test reduces to a single axis comparison (dot product with an axis vector), making it the cheapest per-node test. Balance guarantees $O(\log n)$ depth.
- **BSP Tree:** Each node requires a full dot product with an arbitrary normal, which is slightly more expensive per node than a single axis comparison. The median-projection split guarantees balance, so traversal depth is bounded at $O(\log n)$; however the higher per-node cost still makes it slower than the k-d tree in practice.

For ray casting, the **k-d tree** is generally the best choice: its balance guarantee minimizes tree depth, and axis-aligned planes minimize per-node cost.

5.4 Dynamic Scenes

(Both authors)

For dynamic scenes with moving objects or streaming LiDAR, we would use an **Octree**. Its regular structure makes insertion and deletion local operations: a moving point only modifies the nodes on its root-to-leaf path, and split planes never need updating. Unlike the k-d tree, which typically requires periodic full rebuilds to restore balance after many insertions, the octree supports truly incremental updates with no rebalancing step. The BSP tree's data-dependent planes make any update expensive since the PCA result changes when the point distribution shifts, potentially invalidating the split planes of entire subtrees.

5.5 Urban Scenes

(Both authors)

For urban scenes we would use an **Octree**. While the k-d tree is also a valid choice – its median split adapts well to non-uniform point densities – the octree has two practical advantages that matter specifically for urban LiDAR data.

First, urban point clouds are extremely large (millions of points) and spatially non-uniform: very dense near building facades and the ground plane, almost empty in open air. A sparse octree compresses this empty space implicitly and at multiple resolutions simultaneously, making it straightforward to implement level-of-detail rendering by simply querying a shallower depth. The k-d tree does not have a natural notion of uniform resolution levels, since its cuboids vary in size and shape across the tree.

Second, urban mapping applications typically require repeated spatial queries of the same type (e.g. nearest-neighbour search, frustum culling, occupancy checks), all of which benefit from the octree's predictable, regular cell structure. Practical LiDAR mapping tools such as OctoMap confirm this choice as the industry standard for large-scale outdoor scenes.

6 Conclusion

(Both authors)

We implemented three spatial partitioning structures – Octree, k-d Tree, and BSP Tree – and integrated them into a single interactive browser-based viewer for LiDAR point clouds, built with Three.js rather than raw OpenGL. This allowed us to focus on algorithm design and rendering architecture without low-level graphics boilerplate, at no cost to visual quality or interactivity.

The comparative analysis confirmed that no single tree dominates across all criteria. The **octree** is the most practical general-purpose structure: fast to build, easy to update incrementally, and naturally suited to level-of-detail rendering of large urban scenes. The **k-d tree** is the best choice for ray casting and nearest-neighbour queries, where its balance guarantee and cheap per-node axis test minimize traversal cost. The **BSP tree** produces the most geometry-adaptive partitions, but its high PCA build cost and the fact that any point insertion invalidates the local PCA planes make it less suitable for dynamic or real-time applications.

On the implementation side, three decisions had the largest practical impact. **Replacing `Vector3` arrays with integer indices** during tree construction reduced peak memory for Hasselt from ∼784 MB to ∼78 MB, eliminating out-of-memory crashes and making all three k-d tree split-mode variants cacheable simultaneously. **InstancedMesh rendering** reduced thousands of draw calls to one, keeping the frame rate above 60 fps at deep tree levels where per-node meshes failed entirely. **Tree caching** – building each tree on first use and storing it under a composite (algorithm, dataset, variant) key – was the single most impactful UX decision, making all interactions instant even on the largest tested dataset. A fast-path binary PCD parser was also implemented: for files with the standard `x y z Float32` layout (all three bundled datasets), the position buffer is created directly

from the raw bytes after the header, eliminating ∼58.8 million `DataView` calls for Hasselt and reducing the post-download parsing phase from seconds to milliseconds.

Overall, the project reinforced that the choice of spatial data structure is inseparable from the intended query type, update frequency, and scene characteristics – a lesson that extends well beyond point cloud processing to any geometry-heavy application.